# Retos de programación

## Reto 1: El cajero automático inteligente (cajero.py)

### 1.1 Tu primer algoritmo voraz

En informática, un **algoritmo voraz** es una estrategia de resolución de problemas basada en una regla de oro:

> En cada paso, toma la decisión que parezca mejor en ese instante, sin mirar al futuro ni retroceder. En lugar de calcular y comparar combinaciones complejas, el programa construye la solución de forma directa eligiendo siempre la opción más ventajosa en cada momento.

El ejemplo clásico de esta estrategia es la **retirada óptima de efectivo**: si queremos dar la menor cantidad posible de billetes, la decisión voraz consiste en agotar siempre el billete más grande disponible antes de pasar al siguiente. Con el sistema de euros, esta técnica garantiza siempre la solución óptima.

### 1.2 Tu objetivo

Escribe un programa en Python que simule el **dispensador de billetes de un cajero automático:**

- Pide al usuario la cantidad en euros a retirar. Como el cajero solo dispone de billetes, la cantidad debe ser un **entero positivo múltiplo de 5**. Si el usuario introduce un valor incorrecto (por ejemplo, negativo o que no acabe en 0 o 5), vuelve a pedírselo en un bucle hasta que sea válido.
- El cajero dispone de **billetes de 50 €, 20 €, 10 € y 5 €**. Calcula cuántos billetes de cada tipo debe entregar, priorizando siempre el valor más alto posible para minimizar el total de billetes entregados.
- Muestra un desglose en el que aparezcan únicamente los billetes que realmente se entregan (omite los billetes de cantidad 0) y el número total de billetes suministrados.

**Ejemplo:**

```
Introduce cantidad a retirar (múltiplo de 5): 137
Error: la cantidad debe ser un número positivo múltiplo de 5.

Introduce cantidad a retirar (múltiplo de 5): 135

Billetes entregados:
- 2 billetes de 50 €
- 1 billete de 20 €
- 1 billete de 10 €
- 1 billete de 5 €

Total de billetes entregados: 5
```

**Pistas:**

- Puedes usar un bucle `while` para validar la entrada inicial: `while cantidad % 5 != 0: ...`
- Puedes calcular cuántos billetes caben de 50 con `cantidad // 50` y actualizar el resto con `cantidad %= 50`, o bien hacer un `while cantidad >= 50:` que reste 50 y sume un contador en cada vuelta.

# Reto 2: La conjetura de Collatz (collatz.py)

## 2.1 Sobre la conjetura de Collatz

En 1937, el matemático **Lothar Collatz** propuso un problema que a día de hoy sigue sin resolverse:

- Toma cualquier número entero positivo `n`
- Si es par, divídelo entre 2.
- Si es impar, multiplícalo por 3 y súmale 1.ç
- Repite el proceso con el resultado obtenido.

La conjetura afirma que siempre se llega al número 1, sin importar con qué número empieces.

**¿Por qué sigue sin resolverse?**

Aunque se ha comprobado por ordenador para trillones de números que la secuencia siempre acaba cayendo al 1, nadie ha conseguido demostrar matemáticamente que sea una regla universal. Por eso, sigue siendo una conjetura y no un teorema.

## 2.2 Tu objetivo

Escribe un programa en Python que actúe como banco de pruebas para esta conjetura:

- Pide al usuario un **número entero mayor que 1** (puedes validar con un bucle que no introduzca valores menores o iguales a 1).
- Muestra en una sola línea **la secuencia completa generada hasta llegar al 1**, separando cada número con flechas (->).
- Cuenta y muestra el total de pasos (iteraciones) necesarios para alcanzar el 1.
- Registra y muestra el valor récord alcanzado en todo el trayecto (el "pico" más alto de la secuencia).

**Ejemplo:**

```
Introduce un número inicial: 6

Secuencia: 6 -> 3 -> 10 -> 5 -> 16 -> 8 -> 4 -> 2 -> 1
Pasos totales: 8
Pico más alto alcanzado: 16
```

**Pistas:**

- Usa un bucle `while n != 1:`
- Mantén una variable `maximo` inicializada con el valor inicial de `n` y actualízala dentro del bucle si el nuevo valor supera al anterior.
- Recuerda usar `división entera` para que los números no se conviertan en decimales (float).